Математички методи за машинско учење 2025/2026

Домаћи задатак број 3

Име и презиме студента: Jovan Paunović

Број индекса: 2122

Упутство за израду и предају домаћег задатака:

1. Пре почетка израде промените име датотеке у 03Domaci_Ime_Prezime. (убаците своје име и презиме)
2. Попуните ћелију испод наслова одговарајућим подацима.
1. Употреба ћирилице није обавезна за предају домаћег задатка.
4. За решавање задатака, уколико је потребно, отворите испод текста задатка додатне ћелије за уписивање текстуалног одговора (Markdown) или програмског кода (Code).
1. Сва израчунавања, уколико је потребно, вршити у Python-у.
5. Након завршетка израде решења домаћег Notebook документ сачувати у pdf формату и проследити га наставнику. То можете да урадите или кроз Teams или на мејл адресу jovana.dzunic@elfak.ni.ac.rs

In [96]:
```
import numpy as np
import numpy.random as rndm
import matplotlib as mplb
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_blobs
```

Задатак 1. У стандардном K-means алгоритму, сваки податак припада искључиво једном кластеру. Нека је дат скуп података $X \in \mathbb{R}^{d\times N}$ где је d димензија података, а N број података (подаци су колоне матрице X). Нека је K број жељених кластера. Циљ K-means алгоритма је минимизација функције инерције

$$inercija = \sum_{i=1}^{K} \sum_{x_j \in C_i} \|x_j - c_i\|^2.$$

Нека је $H \in \{0,1\}^{K\times N}$ тзв. индикаторска матрица - бинарна матрица која за сваки податак бележи да ли је елемент кластера,

$$H(i,j) = \begin{cases} 1, & i-\text{том кластеру припада } j-\text{ти податак} \\ 0, & \text{иначе} \end{cases}$$

a) Нека је $C \in \mathbb{R}^{d\times K}$ матрица чије колоне чувају координате центара c_i кластера C_i. У новој итерацији дошло је до прерасподеле података по кластерима, то је садржај ажуриране матрице H. Анализирати структуру матрице $M = HH^T$: Да ли је она пуна, дијагонална или блок матрица? Шта представљају елементи на њеној дијагонали а шта су вандијагонални елементи?

(5 поена)

In [97]:
```
# KxN, K=4 klastera, N=10 podataka
# primer matrice H
H = np.array([[1,0,1,0,0,1,0,0,0,0],
              [0,1,0,0,0,0,1,0,0,0],
              [0,0,0,1,0,0,0,1,1,0],
              [0,0,0,0,1,0,0,0,0,1]])

print(H.T)

M=H@H.T
M
```

```
[[1 0 0 0]
 [0 1 0 0]
 [1 0 0 0]
 [0 0 1 0]
 [0 0 0 1]
 [1 0 0 0]
 [0 1 0 0]
 [0 0 1 0]
 [0 0 1 0]
 [0 0 0 1]]
```

Out[97]:
```
array([[3, 0, 0, 0],
       [0, 2, 0, 0],
       [0, 0, 3, 0],
       [0, 0, 0, 2]])
```

Matrica M je dijagonalna matrica dimenzija $K \times K$. Njeni dijagonalni elementi predstavljaju brojeve elemenata po klasterima. Element $M_{i,i}$ predstavlja broj elemenata u i-tom klasteru. Vandijagonalni elementi su uvek 0, zato što jedan element pripada isključivo jednom klasteru, odnosno u matrici H u jednoj koloni postoji samo jedna jedinica. Zbog toga kad se računa HH^T vandijagonalni elementi ispadaju nule.

5/5поена

б) Потребно је одредити нове центре кластера као центре скупа вектора по кластерима. Показати (извођењем или образложењем) да се координате нових центара кластера могу добити решавањем матричне једначине по C :

$$C(HH^T) = XH^T \quad \iff \quad CM = XH^T.$$

(5 поена)

Novi centar klastera predstavlja centar skupa vektora u tom klasteru:

$$c_i = \frac{1}{n_i} \sum_{x_j \in C_i} x_j \quad (1)$$

pri čemu je n_i broj vektora u i-tom klasteru.

Matrica XH^T u i-toj koloni ima zbir svih vektora koji pripadaju klasteru C_i.

Formulu za izračunavanje centra klastera možemo da zapišemo kao

$$n_i c_i = \sum_{x_j \in C_i} x_j \quad (2)$$

Dakle, na desnoj strani imamo zbir koji odgovara i-toj koloni matrice XH^T.

Na levoj strani imamo matricu centara klastera koja se množi matricom M kod koje element $M_{i,i} = n_i$ predstavlja broj elemenata u klasteru C_i. Rezultat ovog proizvoda je matrica dimenzija $d \times K$ (što se poklapa sa dimenzijama matrice XH^T), koja u i-toj koloni sadrži proizvod $n_i c_i$, što je na osnovu formule (2) zapravo jednako sumi vektora koji pripadaju klasteru C_i, a to je upravo i-ta kolona matrice XH^T. Odatle zaključujemo da važi jednakost

$$CM = XH^T$$

Kako bismo odredili matricu C odnosno nove centre klastera, potrebno je rešiti ovu matričnu jednačinu po C. To se postiže sledećim izrazom:

$$C = XH^T M^{-1}$$

Bitno je napomenuti da matrica M ima inverznu matricu samo ukoliko svaki klaster ima bar po jedan element.

Ovo ćemo proveriti sledećim Python kodom:

In [98]:
```
# d=2, N=4
X = np.array([[1, 2, 0, 0],
              [0, 0, 2, 3]])

# prvi klaster ima 3 elementa, drugi ima 1
H = np.array([[1, 1, 1, 0],
              [0, 0, 0, 1]])

M = H @ H.T
XHT = X @ H.T

M_inv = np.linalg.inv(M)

C = XHT @ M_inv

# racunamo centre rucno radi provere
cluster1 = X[:, 0:3]
cluster2 = X[:, 3:]
mean1 = np.mean(cluster1, axis=1)
mean2 = np.mean(cluster2, axis=1)

print("Matrica M = H H^T:\n", M)
print("Matrica X H^T:\n", XHT)
print("Matrica C\n", C)
print("Centar klastera 1 (očekivano):", mean1)
print("Centar klastera 2 (očekivano):", mean2)
```

```
Matrica M = H H^T:
 [[3 0]
 [0 1]]
Matrica X H^T:
 [[3 0]
 [2 3]]
Matrica C
 [[1.         0.        ]
 [0.66666667 3.        ]]
Centar klastera 1 (očekivano): [1.         0.66666667]
Centar klastera 2 (očekivano): [0. 3.]
```

Možemo zaključiti da se dobijeni rezultati slažu, što potvrđuje činjenicu da se novi centri klastera, odnosno matrica C može odrediti na ranije opisani način.

5/5поена

в) У напреднијим алгоритмима кластеризације (нпр. *Gaussian Mixture Models* или *Fuzzy C-means*) информације о припадности кластеру нису "црно-беле" већ носе опис могућности припадања једног податка свим кластерима. То се најчешће изражава вероватноћом припадности елемента кластерима. Индикаторска матрица је сада $H \in [0,1]^{K\times N}$ са својством да је збир елемената дуж сваке колоне једнак 1,

$$H(i,j) = \text{вероватноћа да } j-\text{ти податак припада } i-\text{том кластеру} \in [0,1].$$

$$\sum_{i=1}^{K} H_{ij} = 1 \text{ за свако } j$$

Претпоставка је да ниједан од кластера није без елемената, тј. да су вероватноће свих елемената за њега једнаке 0.

Анализирати структуру матрице $M = HH^T$: Да ли је она пуна, дијагонална или блок матрица? Шта представљају елементи на њеној дијагонали а шта су вандијагонални елементи?

(5 поена)

In [99]:
```
# KxN, K=4 klastera, N=10 podataka
# primer matrice H
H = np.array([[0.8, 0.1, 0.7, 0.2, 0.0, 0.9, 0.1, 0.2, 0.3, 0.0],
              [0.1, 0.7, 0.1, 0.1, 0.0, 0.05, 0.8, 0.1, 0.2, 0.0],
              [0.05, 0.1, 0.1, 0.6, 0.1, 0.02, 0.05, 0.6, 0.4, 0.2],
              [0.05, 0.1, 0.1, 0.1, 0.9, 0.03, 0.05, 0.1, 0.1, 0.8]])

print(H.T)

M=H@H.T
M
```

```
[[0.8  0.1  0.05 0.05]
 [0.1  0.7  0.1  0.1 ]
 [0.7  0.1  0.1  0.1 ]
 [0.2  0.1  0.6  0.1 ]
 [0.   0.   0.1  0.9 ]
 [0.9  0.05 0.02 0.03]
 [0.1  0.8  0.05 0.05]
 [0.2  0.1  0.6  0.1 ]
 [0.3  0.2  0.4  0.1 ]
 [0.   0.   0.2  0.8 ]]
```

Out[99]:
```
array([[2.13  , 0.445 , 0.503 , 0.222 ],
       [0.445 , 1.2125, 0.326 , 0.1665],
       [0.503 , 0.326 , 0.9554, 0.4356],
       [0.222 , 0.1665, 0.4356, 1.5059]])
```

Matrica M je sada puna, a ne dijagonalna kao u delu pod (a). Takođe je i simetrična.

Dijagonalni elementi $M_{i,i}$ su definisani na sledeći način:

$$M_{i,i} = \sum_{j=1}^{N} H(i,j)^2$$

To je zbir kvadrata verovatnoća da svaki podatak pripada klasteru i. Praktično, što su veće verovatnoće pripadnosti podataka klasteru i, to je ovaj zbir veći, pa može da se tumači kao neka veličina klastera, pri čemu ne daje jasno definisan broj elemenata u klasteru.

Vandijagonalni elementi $M_{p,q},\ p \neq q$ su definisani na sledeći način:

$$M_{p,q} = \sum_{j=1}^{N} H(p,j)\, H(q,j)$$

Praktično se računa skalarni proizvod između vrsta p i q u matrici H. Ovi elementi pokazuju koliko su klasteri p i q „slični“ u smislu da isti podaci imaju nenulte verovatnoće pripadnosti i jednom i drugom klasteru. Veliki su ako postoji veliki broj elemenata koji imaju značajne verovatnoće pripadnosti oba klastera p i q. Ako su pripadnosti „stroge“ (kao u delu pod (a)), ovi elementi bi bili

5/5поена

г) У овом типу кластеровања израчунавање центара кластера више није проста аритметичка средина вектора из самог кластера већ тежинска средња вредност,

$$c_i = \frac{\sum_{j=1}^{N} H_{ij} x_j}{\sum_{j=1}^{N} H_{ij}}.$$

Уводи се дијагонална матрица тежина:

$$D = \operatorname{diag}\Big(\sum_j H_{1j}, \dots, \sum_j H_{Kj} \Big),$$

Показати (извођењем или образложењем) да се координате нових центара кластера могу и сада добити решавањем матричне једначине по C :

$$CD = XH^T.$$

(5 поена)

Formula za računanje težinske srednje vrednosti praktično znači da svaki podatak x_j doprinosi centru klastera c_i proporcionalno svojoj verovatnoći H_{ij} da pripada tom klasteru.

Element $D_{i,i}$ matrice D predstavlja sumu verovatnoća pripadnosti podataka klasteru i. To je praktično imenilac u formuli za računanje težinske srednje vrednosti. Analogno delu pod (a), formulu za računanje težinske srednje vrednosti možemo da zapišemo i na sledeći način:

$$c_i \cdot \left(\sum_{j=1}^{N} H_{ij} \right) = \sum_{j=1}^{N} H_{ij} x_j$$

Sada posmatramo matricu proizvoda XH^T. Njena i-ta kolona je:

$$(XH^T)_{:,i} = \sum_{j=1}^{N} x_j H_{ij}$$

što je zapravo brojilac iz formule za računanje težinske srednje vrednosti, odnosno predstavlja težinski zbir vektora za klaster i (svaki vektor iz skupa podataka pomnožen verovatnoćom da pripada klasteru i, pa sve sabrano).

S obzirom na to da je:

$$D = \text{diag}\Big(\sum_j H_{1j}, \dots, \sum_j H_{Kj} \Big),$$

onda izraz

$$c_i \cdot \left(\sum_{j=1}^{N} H_{ij} \right)$$

može matrično da se zapiše kao CD.

Zbog svega toga važi sledeća jednačina:

$$CD = XH^T$$

jer se po kolonama dobija:

$$c_i \cdot D_{ii} = \sum_{j=1}^{N} x_j H_{ij}$$

Rešavanjem po matrici C, dobijamo:

$$C = XH^T D^{-1}$$

što znači da se centri klastera dobijaju kao težinske srednje vrednosti, u skladu sa verovatnoćama pripadnosti.

Takođe, kao i u delu (a), možemo ovo proveriti sledećim Python kodom:

In [100]:
```
# d=2, N=4
X = np.array([[1, 2, 0, 0],
              [0, 0, 2, 3]])

# matrica H sada sadrži verovatnoće pripadnosti klasterima
H = np.array([[0.8, 0.6, 0.7, 0.2],
              [0.2, 0.4, 0.3, 0.8]])

D = np.diag(np.sum(H, axis=1))

XHT = X @ H.T

D_inv = np.linalg.inv(D)
C = XHT @ D_inv


weights1 = H[0, :]
weights2 = H[1, :]

# rucna provera
mean1 = (X @ weights1) / np.sum(weights1)
mean2 = (X @ weights2) / np.sum(weights2)

print("Matrica D:\n", D)
print("Matrica X H^T:\n", XHT)
print("Matrica C:\n", C)
print("Centar klastera 1 (očekivano):", mean1)
print("Centar klastera 2 (očekivano):", mean2)
```

```
Matrica D:
 [[2.3 0. ]
 [0.  1.7]]
Matrica X H^T:
 [[2. 1.]
 [2. 3.]]
Matrica C:
 [[0.86956522 0.58823529]
 [0.86956522 1.76470588]]
Centar klastera 1 (očekivano): [0.86956522 0.86956522]
Centar klastera 2 (očekivano): [0.58823529 1.76470588]
```

5/5поена

д) Сада је потребно да се симулира поступак кластеризације у коме се припадност тачака кластерима одређује преко **вероватноћа** према правилима описаним у претходном делу задатака.

1. **Генерисање података**
   Креирати 2D скуп података коришћењем функције `make_blobs`.

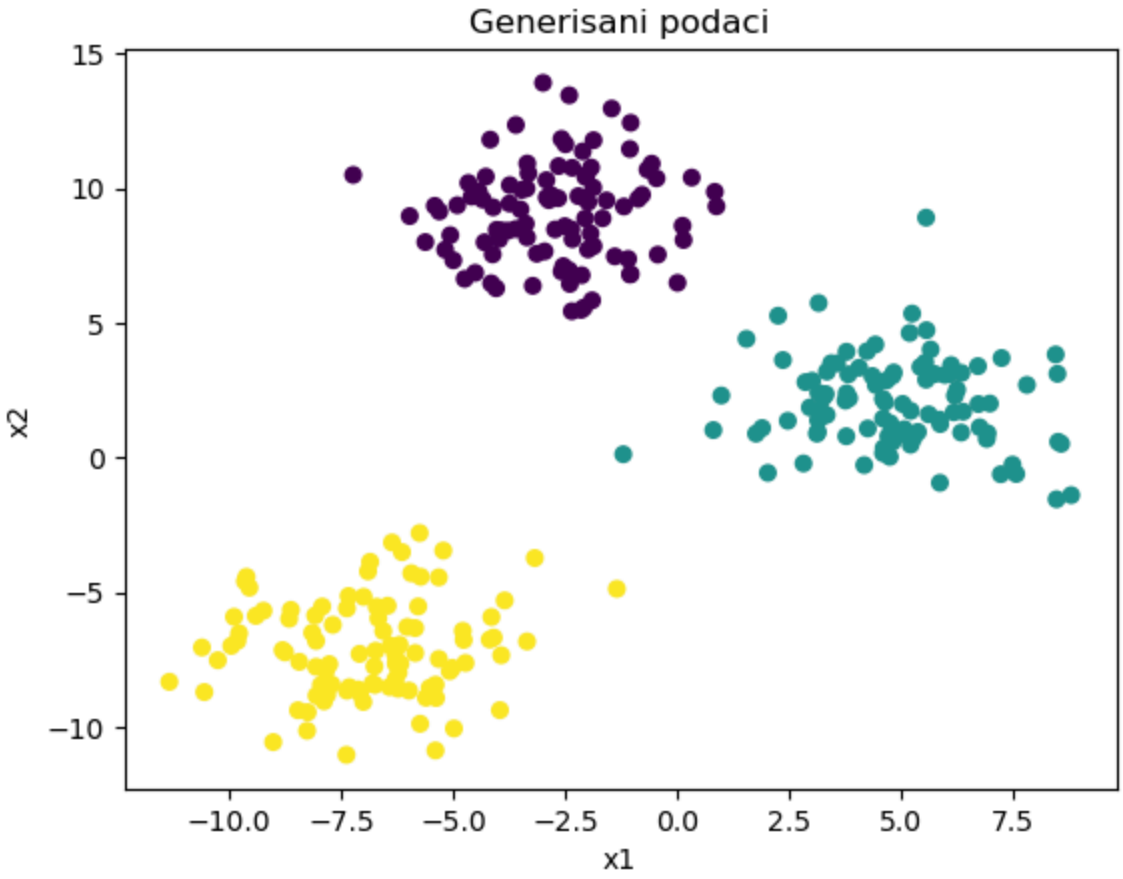
   - Број кластера: 3
   - Број тачака: 300

- Стандардна девијација кластера по избору
- Опционо: насумично seed ради репродуктивности резултат

(2 поена)

In [101]:
```
X, y_true = make_blobs(n_samples=300, n_features=2, centers=3, cluster_s

plt.scatter(X[:, 0], X[:, 1], c=y_true, cmap='viridis', s=30)
plt.title("Generisani podaci")
plt.xlabel("x1")
plt.ylabel("x2")
plt.show()
```



2/2поена

2. **Иницијализација матрице центара**
   Иницијализовати матрицу центара

$$C^{(0)} \in \mathbb{R}^{2\times 3}$$

насумичним избором три тачке из скупа података или униформно из опсега података.

(3 поена)

In [102]:
```
np.random.seed(42)
indices = np.random.choice(X.shape[0], size=3, replace=False)

C0 = X[indices].T

print("Inicijalna matrica centara:\n", C0)
```

```
Inicijalna matrica centara:
 [[ 3.57902227 -6.66309505  5.1456642 ]
 [ 3.50245346 -5.95411969  0.85231055]]
```

3/3поена

3. **Итеративно ажурирање (5 итерација)**
   Извршити укупно **5 итерација** поступка.

   У свакој итерацији:

- Израчунавање матрице припадности

  Прво се израчуна привремена матрица $\tilde{H}^{(t)}$ према инверзним квадратима растојања (са малим ε ради стабилности):

$$\tilde{H}_{ij}^{(t)} = \frac{1}{\|x_j - c_i^{(t)}\|^2 + \varepsilon}, \qquad \varepsilon = 10^{-8}.$$

  Затим се нормализује по колонама да би свака колона имала збир једнак 1 :

$$H_{ij}^{(t)} = \frac{\tilde{H}_{ij}^{(t)}}{\sum_{p=1}^{K} \tilde{H}_{pj}^{(t)}}.$$

  Ово гарантује да је $H^{(t)} \in [0,1]^{K\times N}$ и да важи

$$\sum_{i=1}^{K} H_{ij}^{(t)} = 1 \quad \text{за свако } j.$$

- Ажурирајте матрицу центара коришћењем формуле:

$$C^{(t+1)} = X H^{(t)T} \left(H^{(t)} H^{(t)T} \right)^{-1}.$$

(5 поена)

In [103]:

```
if X.shape[0] != 2: # if obezbedjuje da se X samo jednom transponuje, ak
    X = X.T # prilagodjavanje oblika da se uklapa sa ranijim opisom tj.

K = 3
N = X.shape[1] # 300
eps = 1e-8
C=C0.copy() # 2x3 tj. dxK

def do_iteration(C):
    # X.T[None, :, :] ima dimenzije (1,300,2)
    # C.T[:, None, :] ima dimenzije (3,1,2)
    # zbog broadcastinga, kad se oduzmu dobija se rezultat oblika (3, 30
    # racunamo normu duz trece ose, kvadriramo i dobijamo matricu D dime
    # element Dij predstavlja udaljenost tacke j od centra klastera i
    D = np.linalg.norm(X.T[None,:,:] - C.T[:,None,:], axis=2)**2

    H_tilde = 1.0 / (D + eps)
    H = H_tilde / np.sum(H_tilde, axis=0)

    HHT_inv = np.linalg.inv(H @ H.T)
    C = X @ H.T @ HHT_inv

    return C, H

for t in range(5):
    C, _ = do_iteration(C)

    print(f"Iteracija {t+1} - Centri klastera:\n{C}\n")
```

```
Iteracija 1 - Centri klastera:
[[-0.96804232 -8.06262295  5.95022431]
 [11.12659036 -7.17527085  0.87621218]]

Iteracija 2 - Centri klastera:
[[-3.12488319 -7.56980677  5.8518515 ]
 [10.64279167 -7.80350772  1.55067384]]

Iteracija 3 - Centri klastera:
[[-3.19344844 -7.468128    5.70229603]
 [10.14548655 -7.83978226  1.79220975]]

Iteracija 4 - Centri klastera:
[[-3.1667244  -7.45374548  5.67385138]
 [ 9.99209044 -7.85529702  1.8186586 ]]

Iteracija 5 - Centri klastera:
[[-3.15183249 -7.45412029  5.67460996]
 [ 9.94655028 -7.86240904  1.81784544]]
```

5/5поена

4. **Приказ резултата**

- Након сваке итерације нацртати:
  - тачке података у 2D равни,
  - тренутно израчунате центре,
  - боју тачака одредити према кластеру са највећом вероватноћом.
- По завршетку 5 итерација приказати завршне центре.

(5 поена)

In [104]:

```
C=C0.copy()
for t in range(5):
    C, H = do_iteration(C)

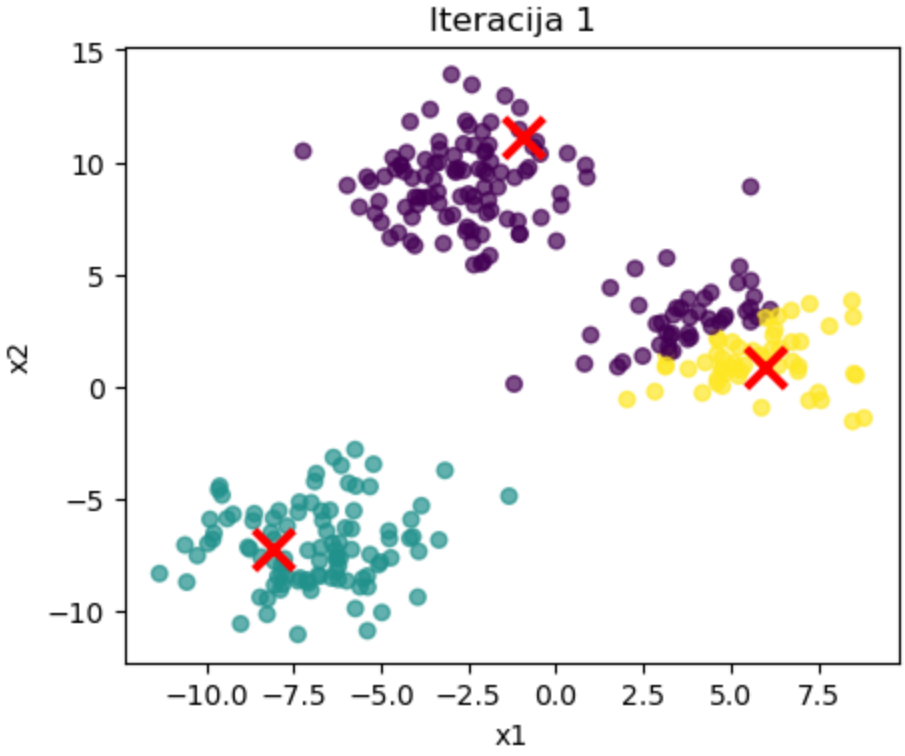
    print(f"Iteracija {t+1} - Centri klastera:\n{C}\n")

    labels = np.argmax(H, axis=0)

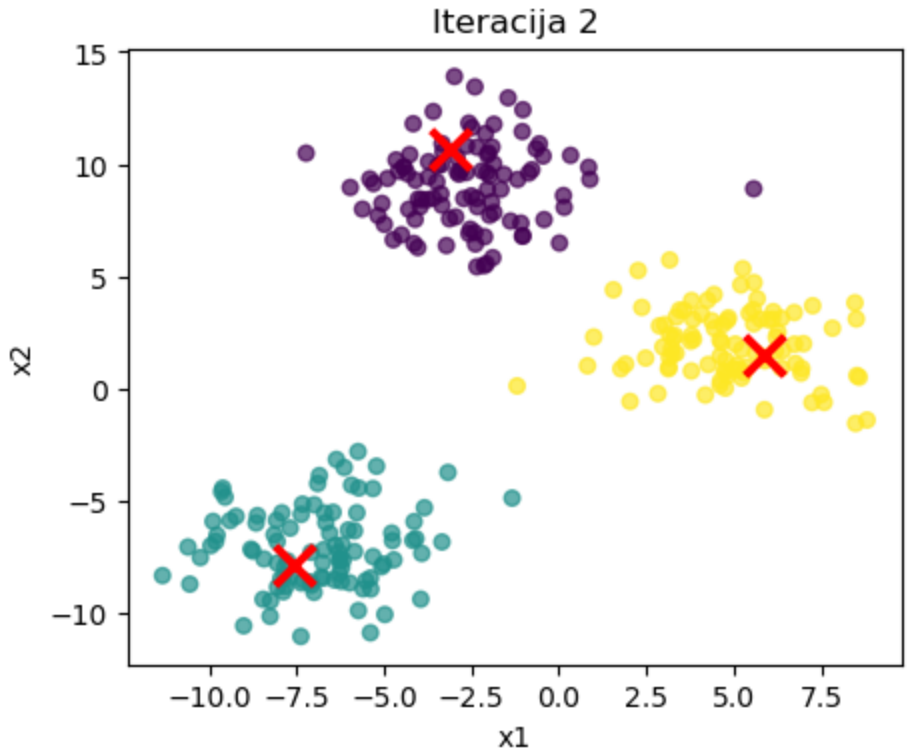
    plt.figure(figsize=(5, 4))
    plt.scatter(X[0, :], X[1, :], c=labels, cmap='viridis', s=30, alpha=
    plt.scatter(C[0, :], C[1, :], c='red', marker='x', s=200, linewidths
    plt.title(f"Iteracija {t+1}")
    plt.xlabel("x1")
    plt.ylabel("x2")
    plt.show()

C5 = C.copy()
```

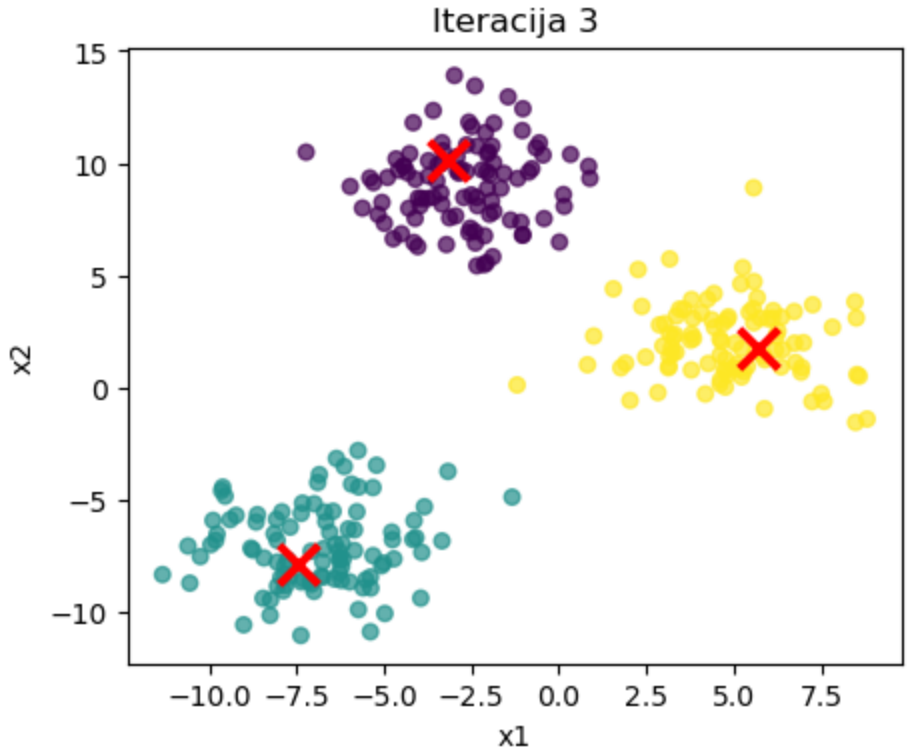
```
Iteracija 1 - Centri klastera:
[[-0.96804232 -8.06262295  5.95022431]
 [11.12659036 -7.17527085  0.87621218]]
```



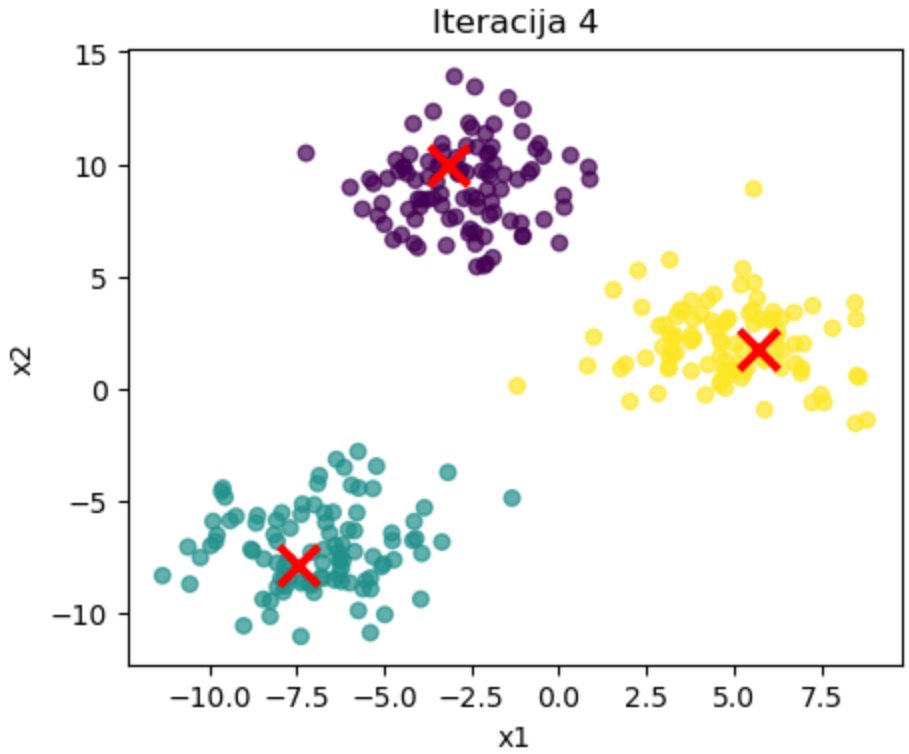

```
Iteracija 2 - Centri klastera:
[[-3.12488319 -7.56980677  5.8518515 ]
 [10.64279167 -7.80350772  1.55067384]]
```



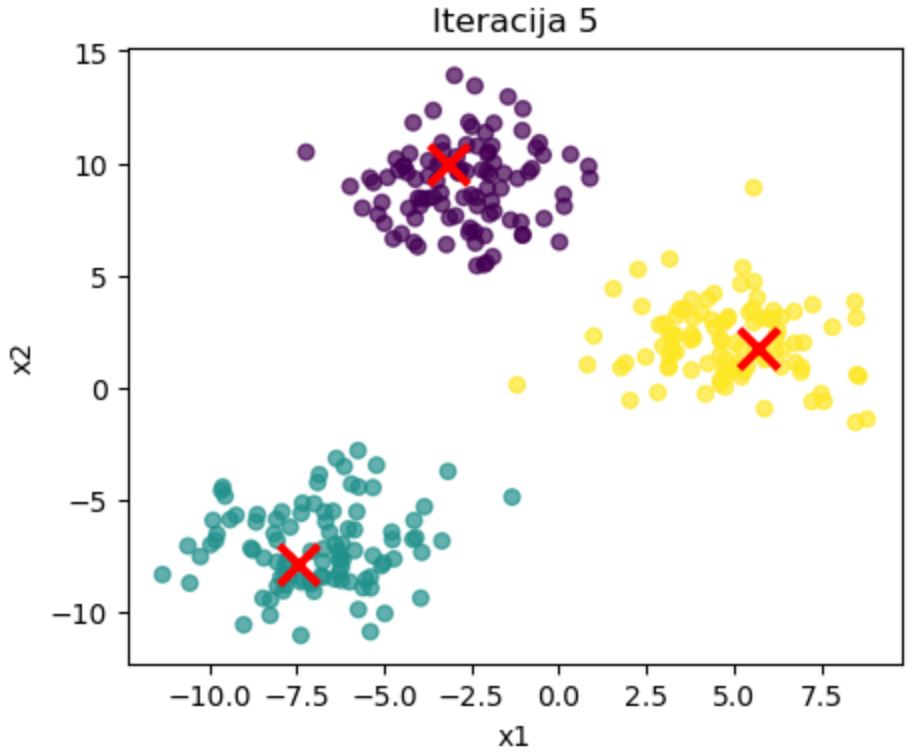

```
Iteracija 3 - Centri klastera:
[[-3.19344844 -7.468128    5.70229603]
 [10.14548655 -7.83978226  1.79220975]]
```




```
Iteracija 4 - Centri klastera:
[[-3.1667244  -7.45374548  5.67385138]
 [ 9.99209044 -7.85529702  1.8186586 ]]
```




```
Iteracija 5 - Centri klastera:
[[-3.15183249 -7.45412029  5.67460996]
 [ 9.94655028 -7.86240904  1.81784544]]
```



5/5поена

5. **Конвергенција**

Након извршених 5 итерација, потребно је **додатно покренути алгоритам** све док се не испуни критеријум конвергенције:

$$\max_i \|c_i^{(t+1)} - c_i^{(t)}\| < 10^{-4},$$

или док се не достигне максималан број итерација (`max_iter = 500`).

Треба коментаристаи:

- укупан број итерација до конвергенције,
- да ли се положаји центара након првих 5 итерација значајно разликују од стања после конвергенције,
- кратак коментар о стабилности и понашању алгоритма; Да ли се центри брзо стабилизују? Да ли је алгоритам осетљив на почетне центре? Да ли су вероватноће понекад „превише меке“ или „превише оштре“? Да ли се центри "здраво" распоређују или склизну у неку регију? Да ли се међусобно приближавају?

(10 поена)

In [106]:

```
tol = 1e-4
max_iter = 500
iter_count = 5 # vec je izvrseno 5 iteracija

for t in range(max_iter - iter_count):
    C_new, H = do_iteration(C)

    diff = np.linalg.norm(C_new - C, axis=0)

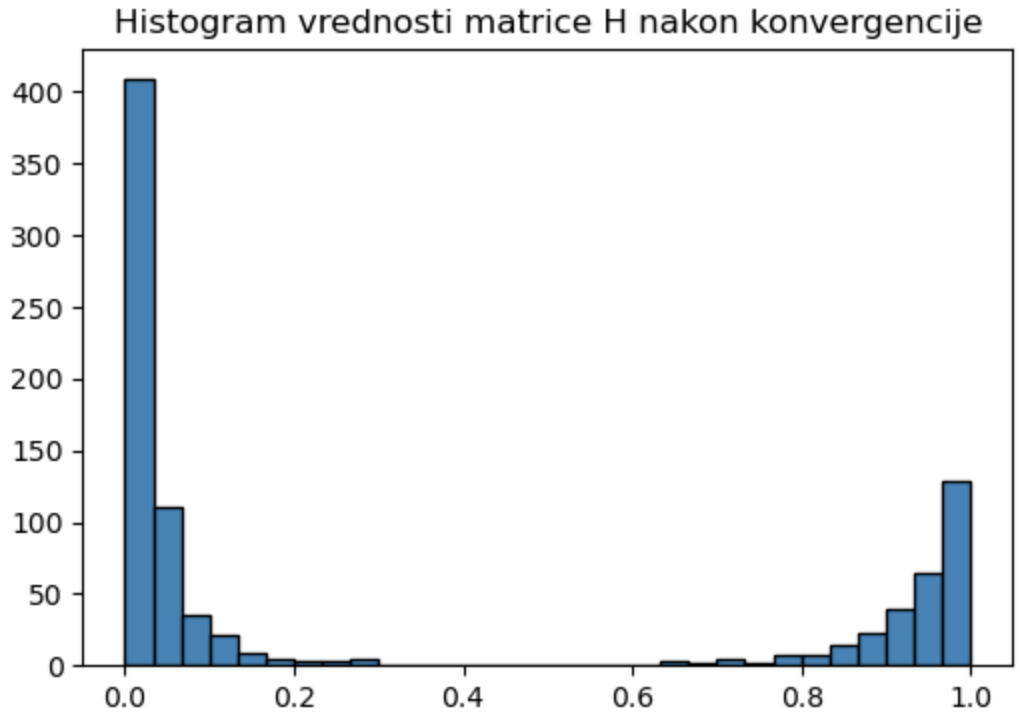
    C = C_new
    iter_count += 1

    if np.max(diff) < tol:
        print(f"Algoritam konvergirao posle ukupno {iter_count} iteracij
        break
else:
    print("Dostignut maksimalan broj iteracija bez konvergencije.")

print("Konacni centri:\n", C)
print("Centri nakon 5. iteracije:\n", C5)
print("Razlika izmedju konacnih i centara nakon 5. iteracije:\n", C-C5)

plt.figure(figsize=(6,4))
plt.hist(H.flatten(), bins=30, color='steelblue', edgecolor='black')
plt.title("Histogram vrednosti matrice H nakon konvergencije")
plt.show()
```

```
Algoritam konvergirao posle ukupno 6 iteracija.
Konacni centri:
 [[-3.14265425 -7.45615495  5.6807813 ]
 [ 9.92555565 -7.86668877  1.8137383 ]]
Centri nakon 5. iteracije:
 [[-3.15183249 -7.45412029  5.67460996]
 [ 9.94655028 -7.86240904  1.81784544]]
Razlika izmedju konacnih i centara nakon 5. iteracije:
 [[ 0.00917825 -0.00203466  0.00617135]
 [-0.02099463 -0.00427973 -0.00410714]]
```



Komentari:

- Ukupan broj iteracija za koje je algoritam konvergirao je 11 (računajući i prvih 5 izvršenih ranije). To je daleko manje od maksimalnih 500 iteracija, što znači da algoritam brzo konvergira.
- Položaji centara nakon 5. iteracije su veoma bliski položajima centara nakon konvergencije. Njihovim poređenjem vidimo da se oni razlikuju najčešće tek na trećoj decimali. Ovo znači da se centri relativno brzo stabilizuju, a dodatne iteracije uglavnom samo fino podešavaju pozicije centara.
- Algoritam se ponaša stabilno, odnosno centri se brzo stabilizuju i ne osciluju. Kao i ostale varijante K-means algoritma, i ovaj algoritam je osetljiv na izbor početnih centara, odnosno broj iteracija koji će biti potreban za konvergenciju algoritma zavisi od izbora početnih centara.
- Verovatnoće pripadnosti podataka klasterima su ponekad previše "oštre", a ponekad i previše "meke". Na primer, trenutno je standardna devijacija klastera u funkciji `make_blobs` postavljena na vrednost 1.8, a sa nacrtanih plotova možemo da vidimo da su klasteri jasno odvojeni. Takođe, verovatnoće u matrici H su oštre, odnosno jasno se može odrediti kom klasteru podatak pripada. Sa histograma možemo da vidimo da su vrednosti verovatnoća najčešće manje od 0.2 ili veće od 0.8. Praktično, ako su klasteri jasno razdvojeni, onda su i centri daleko, pa su i verovatnoće "oštre". Sa druge strane, postavljanjem standardne devijacije klastera na vrednost 3 ili više, klasteri više nisu jasno odvojeni već se preklapaju. U tom slučaju, verovatnoće su više "meke", tj. može se desiti da neka tačka ima skoro podjednake verovatnoće pripadnosti različitim klasterima.
- U normalnim uslovima, centri se raspoređuju "zdravo" oko gustih regiona tačaka i ne približavaju se previše jedan drugom, osim ako su početni centri veoma loše izabrani. U našem slučaju algoritam je dobro klasterizovao podatke.

10/10поена

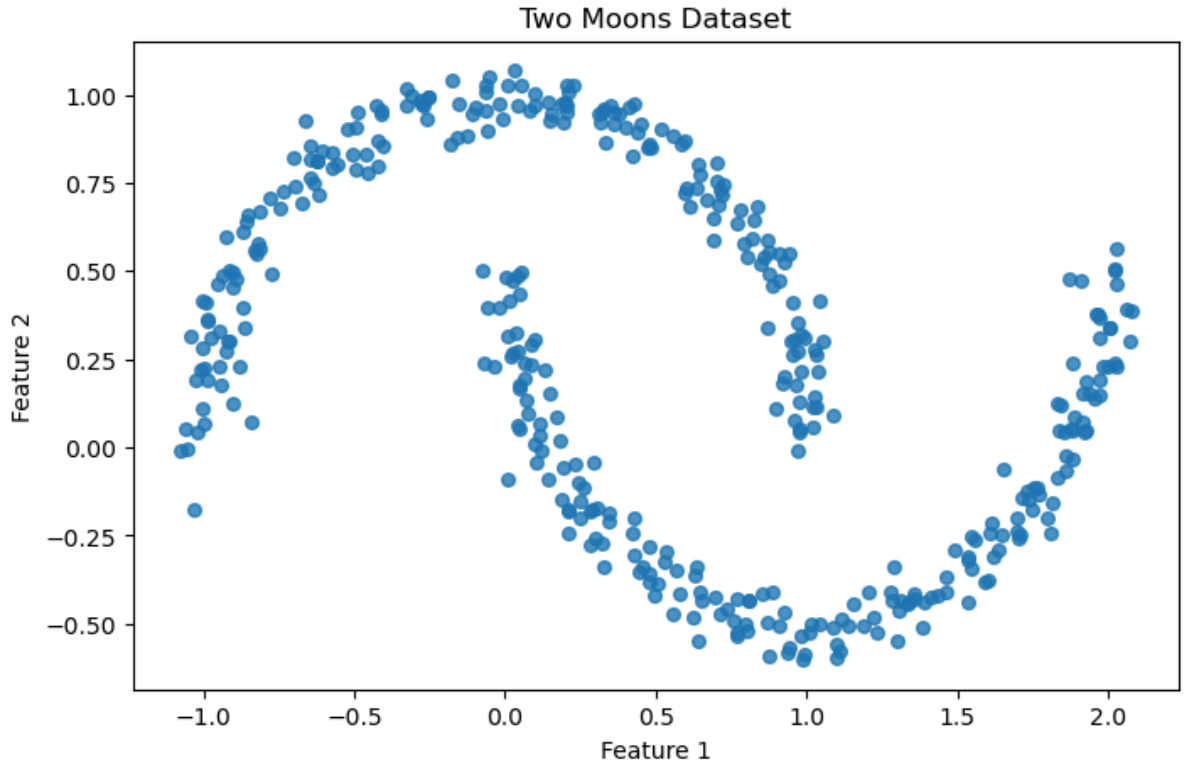
Здадатак 2. K-means алгоритам је моћан и брз, али почива на одређеним геометријским претпоставкама. Овај задатак илуструје шта се дешава када се геометрија података не поклапа са метриком коју алгоритам користи.

Користићемо синтетички скуп података **"Two Moons"** који се састоји од два уланчана полумесеца. Ови подаци нису линеарно раздвојиви.

In [45]:
```
from sklearn.datasets import make_moons
```

In [46]:
```
# Generisanje podataka
X_moons, y_moons = make_moons(n_samples=400, noise=0.05, random_state=42

plt.figure(figsize=(8, 5))
plt.scatter(X_moons[:,0], X_moons[:,1], s=30, alpha=0.8)
plt.title("Two Moons Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2");
```



a) Примена K-means алгоритма

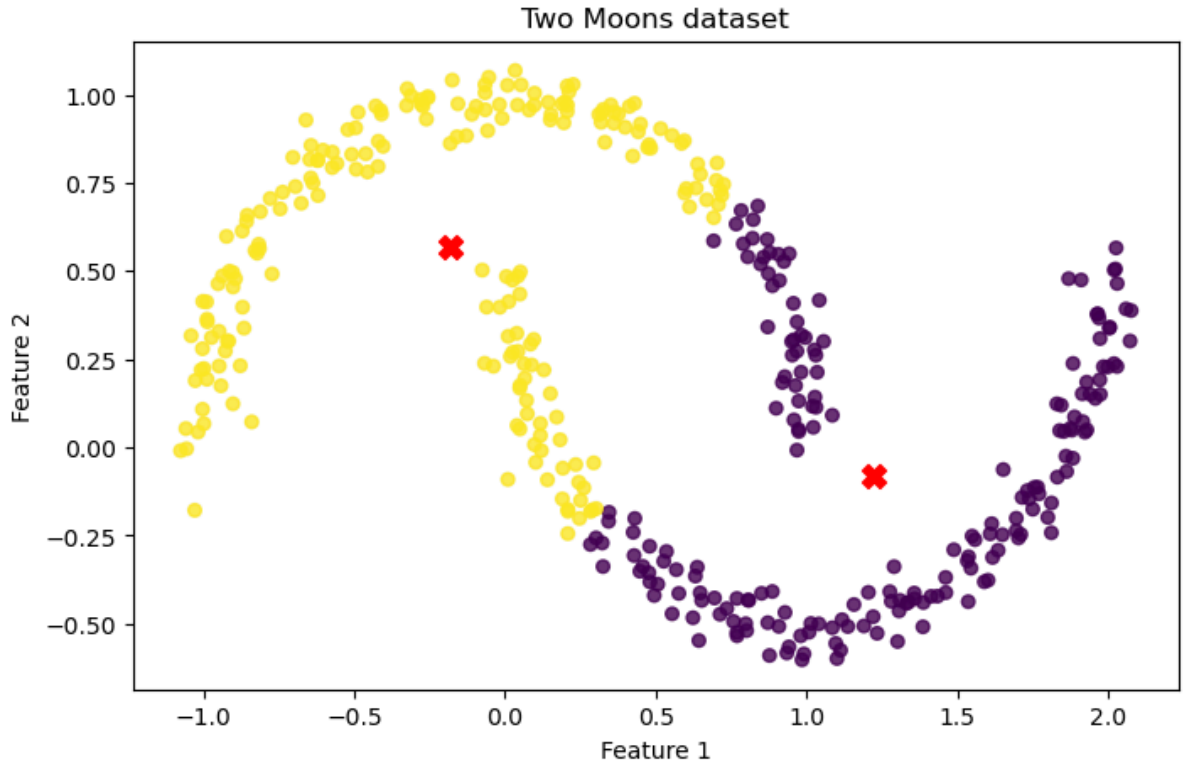
Применити K-means алгоритам са (K=2) над овим скупом података.
Визуелизовати резултат на **Scatter plot-y**:

- Обојити тачке различитим бојама у зависности од додељеног кластера.
- Јасно означити положаје добијених центара (нпр. црвеним `'X'` маркером).

(5 поена)

In [47]:
```
kmeans = KMeans(n_clusters=2, random_state=42)
labels = kmeans.fit_predict(X_moons)
centers = kmeans.cluster_centers_

plt.figure(figsize=(8, 5))
plt.scatter(X_moons[:,0], X_moons[:,1], c=labels, cmap='viridis', s=30,
plt.scatter(centers[:,0], centers[:,1], c='red', marker='X', s=100, line
plt.title("Two Moons dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



5/5поена

б) Резултат који сте добили у претходном кораку вероватно није интуитивно исправан (кластери „секу“ природне облике полумесеца).

Дајте математичко и геометријско објашњење овог феномена одговарајући на следећа питања:

Граница раздвајања кластера (граница одлучивања)

Граница између два кластера чине тачке x које су подједнако удаљене од оба центра c_1 и c_2.

Описано једначином:

$$\|x - c_1\|^2 = \|x - c_2\|^2$$

Питање: Шта геометријски представља ова једначина у $\mathbb{R}^2$?

(2 поена)

Ova jednačina predstavlja pravu u $\mathbb{R}^2$. Konkretno, ukoliko dva dobijena centra spojimo pomoću duži i nacrtamo pravu koja je normalna na ovu duž i deli je na dva jednaka dela, onda je ta prava sastavljena iz tačaka koje su na jednakoj udaljenosti od oba centra. Ta prava predstavlja granicu između dva klastera.

2/2поена

Закључак

Зашто K-means не може да раздвоји ова два полумесеца, без обзира на иницијализацију? Коју геометријску карактеристику показују кластери?

(3 поена)

K-means ne može da razdvoji ova dva polumeseca zato što je granica koju kreira u ovom slučaju prava linija, a podaci iz korišćenog dataseta nisu linearno separabilni u 2D, odnosno nemoguće je razdvojiti ih pravom linijom. Klasteri koji se dobijaju su konveksni, što i jeste odlika klastera dobijenih K-means algoritmom, stoga ovaj algoritam nije adekvatan za klasterizaciju odabranog skupa podataka (Two Moons).

3/3поена

в) Спектрално кластеровање као алтернатива (5 поена)

Проблем „месеца“ се решава променом простора у коме посматрамо податке или променом метрике.

Иако ћемо математику иза овога (сопствене вредности Лапласијана графа) радити касније, овде демонстрирајте решење позивом готове функције.

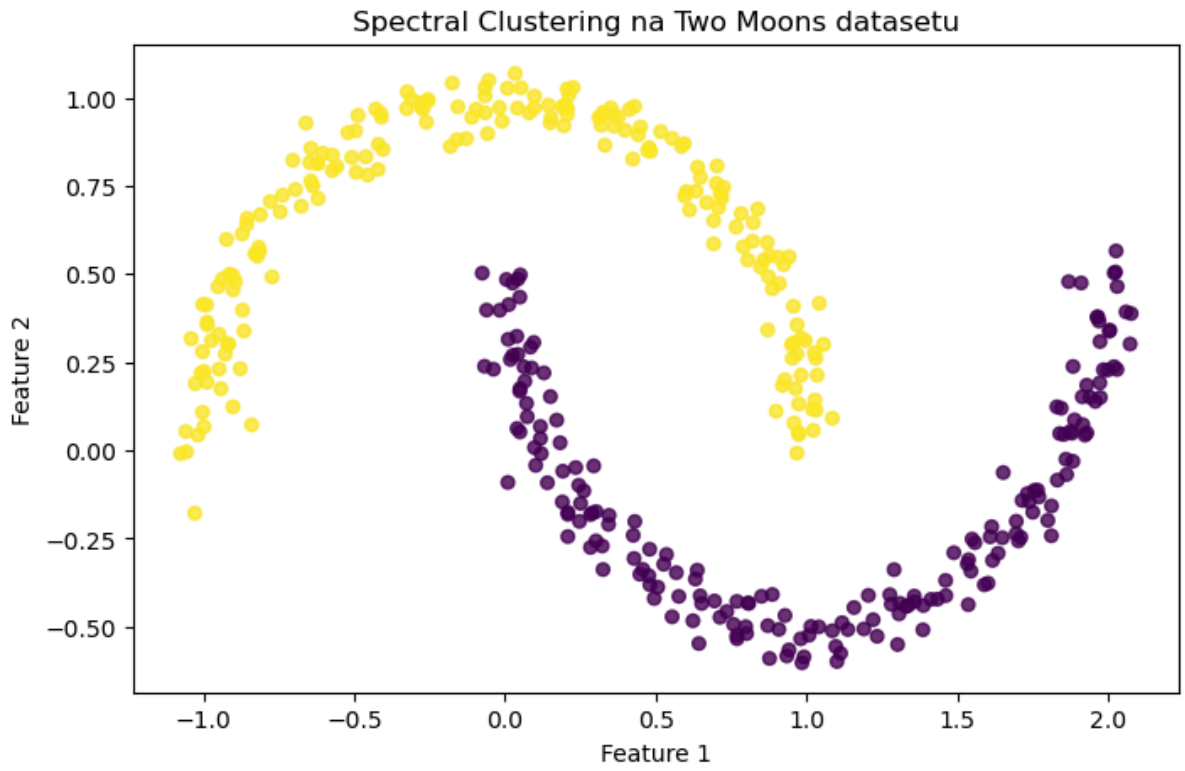
(5 поена)

In [48]:
```
from sklearn.cluster import KMeans, SpectralClustering
```

In [49]:
```
sc = SpectralClustering(n_clusters=2, affinity='nearest_neighbors', rand
labels = sc.fit_predict(X_moons)

plt.figure(figsize=(8, 5))
plt.scatter(X_moons[:,0], X_moons[:,1], c=labels, cmap='viridis', s=30,
plt.title("Spectral Clustering na Two Moons datasetu")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

```
C:\Users\jpaun\miniconda3\envs\faks\Lib\site-packages\sklearn\manifold
\_spectral_embedding.py:328: UserWarning: Graph is not fully connected,
spectral embedding may not work as expected.
  warnings.warn(
```



5/5поена

Задатак 3. Дат је скуп података који садржи информације о карактеристикама производа у продавници: цена производа, тежина производа и оцена корисника. Применом K-means алгоритма групишите производе у кластере. Одредите број кластера користећи мере квалитета кластеризације.

(5 поена)

In [136]:
```
np.random.seed(42) # da bismo svaki put dobili iste rezultate
N=rndm.randint(200,450) #ukupan broj proizvoda
cena=rndm.normal(500,100,N)
tezina=rndm.normal(1000,200,N) #tezina u gramima
ocena=rndm.uniform(1,5,N)
podaci=np.block([[cena],[tezina],[ocena]]).T
podaci.shape
```

Out[136]: (302, 3)

In [140]:

```
def execute_Kmeans(podaci, K_range):
    inertias = []
    sil_scores = []
    labels_dict = {}

    for k in K_range:
        km = KMeans(n_clusters=k, random_state=42, n_init=10)
        labels = km.fit_predict(podaci)
        labels_dict[k] = labels
        inertias.append(km.inertia_)
        sil_scores.append(silhouette_score(podaci, labels, sample_size=1
        print(k)

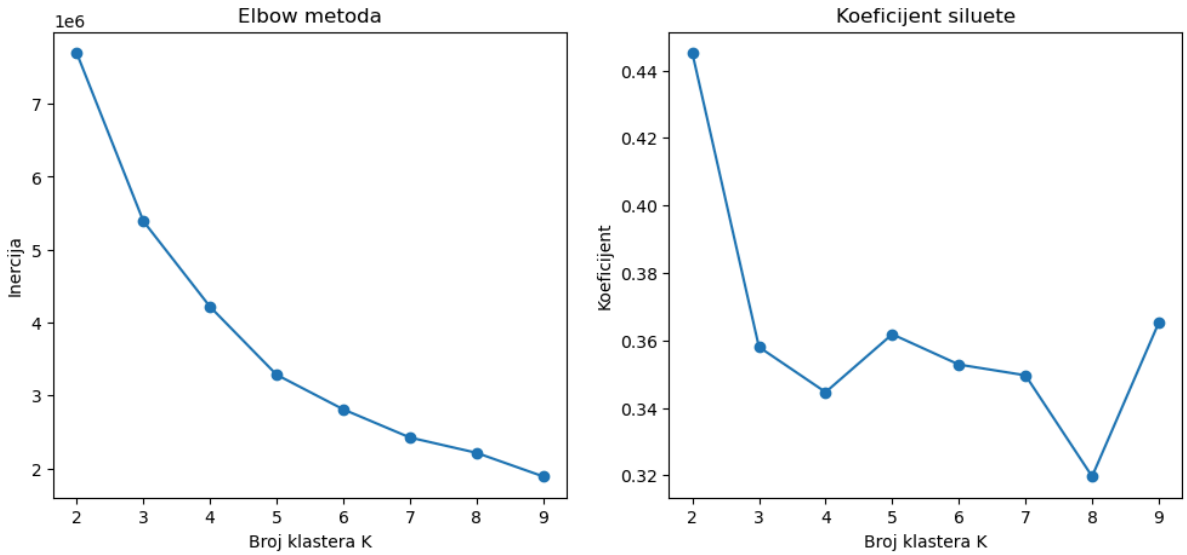
    return inertias, sil_scores, labels_dict

def draw_elbow_and_silhouette(K_range, inertias, sil_scores):
    # Elbow metoda
    plt.figure(figsize=(12,5))
    plt.subplot(1,2,1)
    plt.plot(K_range, inertias, marker='o')
    plt.title("Elbow metoda")
    plt.xlabel("Broj klastera K")
    plt.ylabel("Inercija")

    # Silhouette score
    plt.subplot(1,2,2)
    plt.plot(K_range, sil_scores, marker='o')
    plt.title("Koeficijent siluete")
    plt.xlabel("Broj klastera K")
    plt.ylabel("Koeficijent")
    plt.show()

K_range = np.arange(2, 10)
inertias, sil_scores, labels_dict = execute_Kmeans(podaci, K_range)
draw_elbow_and_silhouette(K_range, inertias, sil_scores)
```

```
2
3
4
5
6
7
8
9
```



Sa prikazanih grafika vidimo da se lakat nazire za $k = 3$ i $k = 5$. Međutim, nakon 3 inercija pada još uvek brzo, dok nakon 5 opada slabije. Takođe, na osnovu koeficijenta siluete potvrđujemo da su ove dve tačke dobar izbor. Obe mere kvaliteta klasterizacije ukazuju na to da je $k = 5$ bolje nego $k = 3$, ali čisto radi analize prikazaćemo rezultate klasterizacije i sa 3 i sa 5 klastera.

5/5поена

За анализу добијених резултата кластеризације можете да користите графички приказ 3D или 2D. За 2D приказ користите парове карактеристика података. Нпр. оцена корисника у функцији цене производа и слично. Припадност различитим кластерима означавате различитим бојама тачака.

(5 поена)

In [141]:

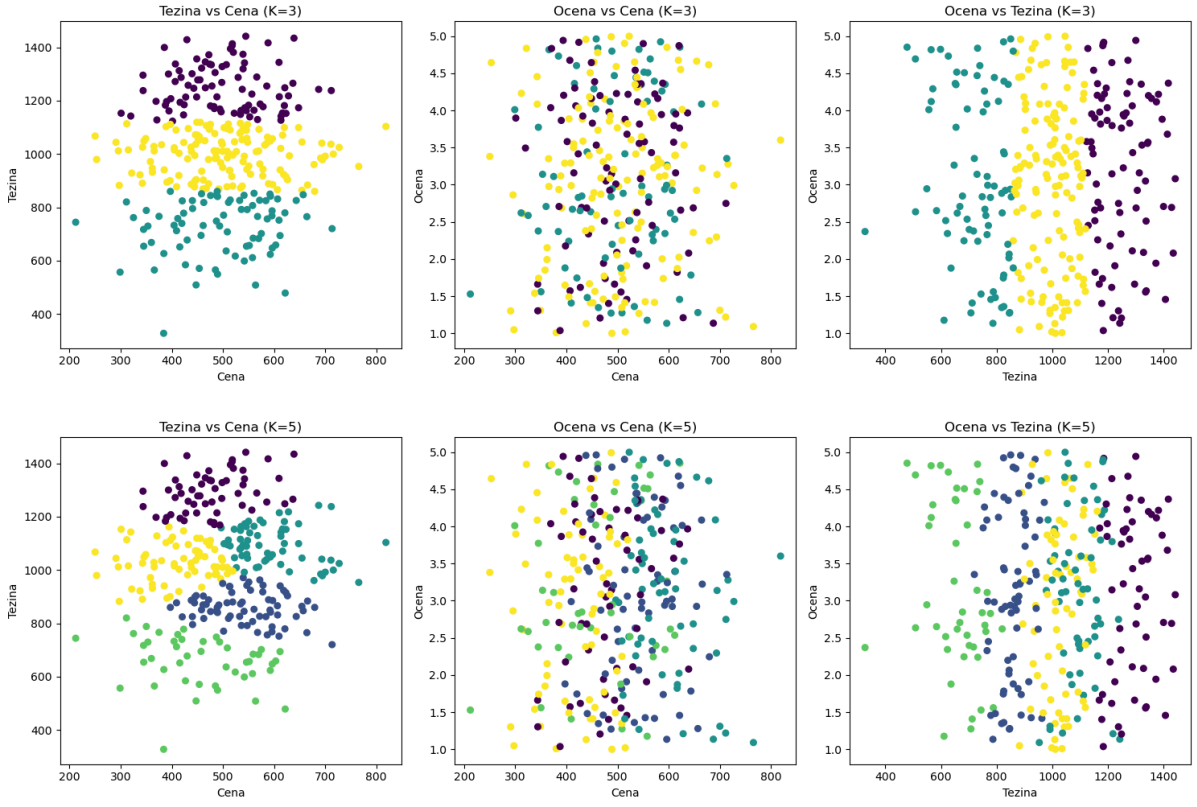
```
def visualize_results(podaci, labels_dict, k):
    features = ["Cena", "Tezina", "Ocena"]
    indices = range(len(features))

    plt.figure(figsize=(15,5))

    plot_id = 1
    for i in range(len(indices)):
        for j in range(i+1, len(indices)):
            plt.subplot(1, 3, plot_id)
            plt.scatter(podaci[:, indices[i]], podaci[:, indices[j]], c=
            plt.xlabel(features[indices[i]])
            plt.ylabel(features[indices[j]])
            plt.title(f"{features[indices[j]]} vs {features[indices[i]]}
            plot_id += 1

    plt.tight_layout()
    plt.show()

visualize_results(podaci, labels_dict, 3)
visualize_results(podaci, labels_dict, 5)
```



Vidimo da težina proizvoda najviše utiče na rezultat klasterizacije. To je i očekivano, zato što su podaci takvi da težina ima najveće vrednosti (oko 1000), a K-means algoritam je osetljiv na redove veličina u podacima, pa "favorizuje" težinu proizvoda. Na scatter-plotu Težina vs Cena podaci su lepo raspodeljeni i za $k = 3$ i $k = 5$, dok je na scatter-plotu Ocena vs Težina bolje rezultate dala klasterizacija sa 3 klastera u smislu da su podaci jasnije razdvojeni.

5/5поена

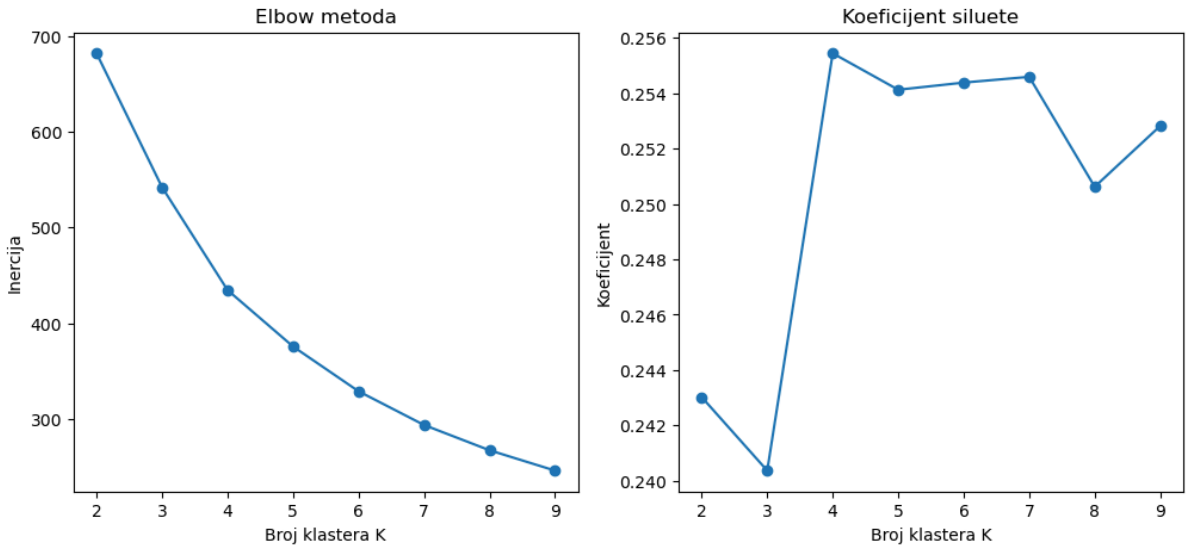
K-means алгоритам је осетљив на редове величина у компонентама података. Скалирајте различите вредности компоненти и проверите резултате примене K-means алгоритма над трансформисаним подацима.

(5 поена)

In [142]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
podaci_scaled = scaler.fit_transform(podaci)

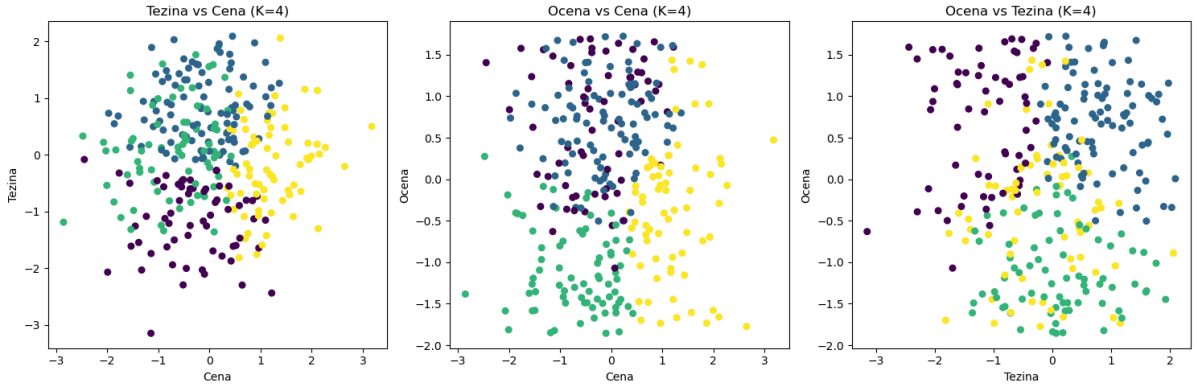
inertias, sil_scores, labels_dict = execute_Kmeans(podaci_scaled, K_rang
draw_elbow_and_silhouette(K_range, inertias, sil_scores)
```

```
2
3
4
5
6
7
8
9
```



Sada je situacija drugačija - vidimo da je optimalan broj klastera $k = 4$ jer je tu i (ne toliko primetan) lakat, a i koeficijent siluete ima maksimum za $k = 4$.

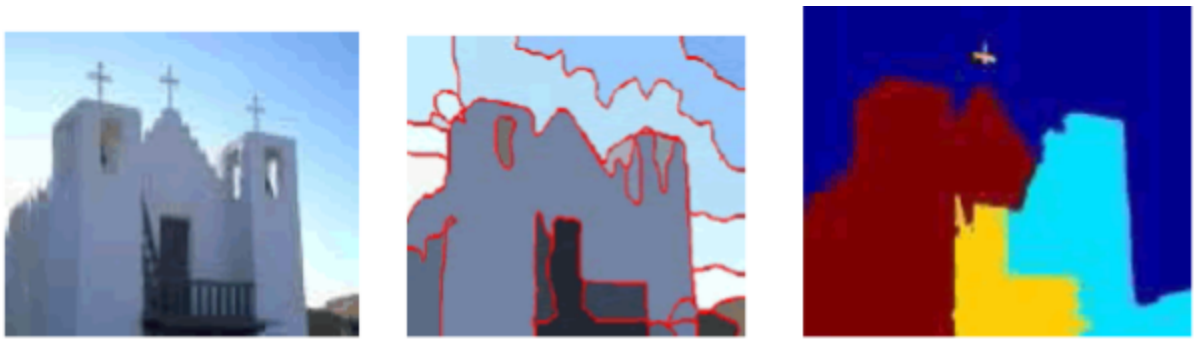
In [145]:
```
visualize_results(podaci_scaled, labels_dict, 4)
```



5/5поена

Задатак 4. За слику учитану наредном кодном ћелијом спровести сегментацију слике у боји употребом K-means алгоритма. Одредити број кластера на основу критеријума ваљаности кластеризације обрађених у радној свесци 15K-means.

Објашњење : Сегментација слике је поступак поделе слике на регионе који су релативно хомогене структуре. Пример слике и сегментиране слике дат је у наставку.



In [21]:
```
import scipy.datasets as scd
rakun=scd.face()
type(rakun)
```

Out[21]: numpy.ndarray

а) За поступак сегментације третирати сваки пиксел као вектор вредности RGB канала. Другим речима сваки пиксел је вектор тродимензионалног простора где одговарајуће компоненте означавају интензитет одговарајућег канала RGB простора слика. Упоредити добијени резултат са одговарајућом сегментацијом ако је слика дата у CIELAB простору слика.

Prvo je potrebno odrediti broj klastera pomoću elbow metode i koeficijenta siluete. U te svrhe će biti korišćene ranije definisane funkcije `execute_Kmeans` i `draw_elbow_and_silhouette`.

In [22]:

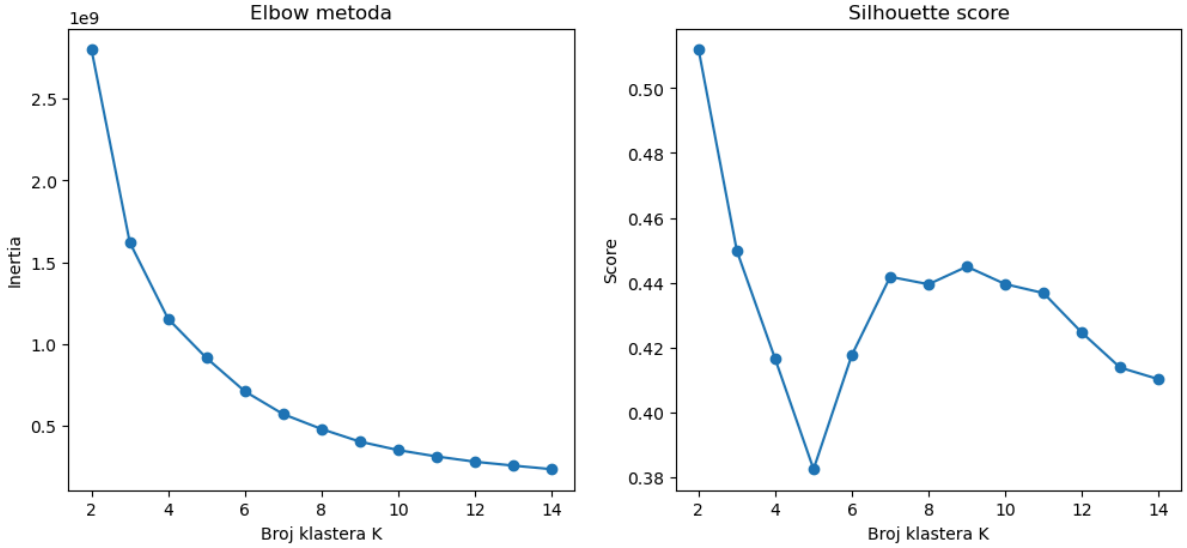
```
from skimage import color

rakun_cielab = color.rgb2lab(rakun)

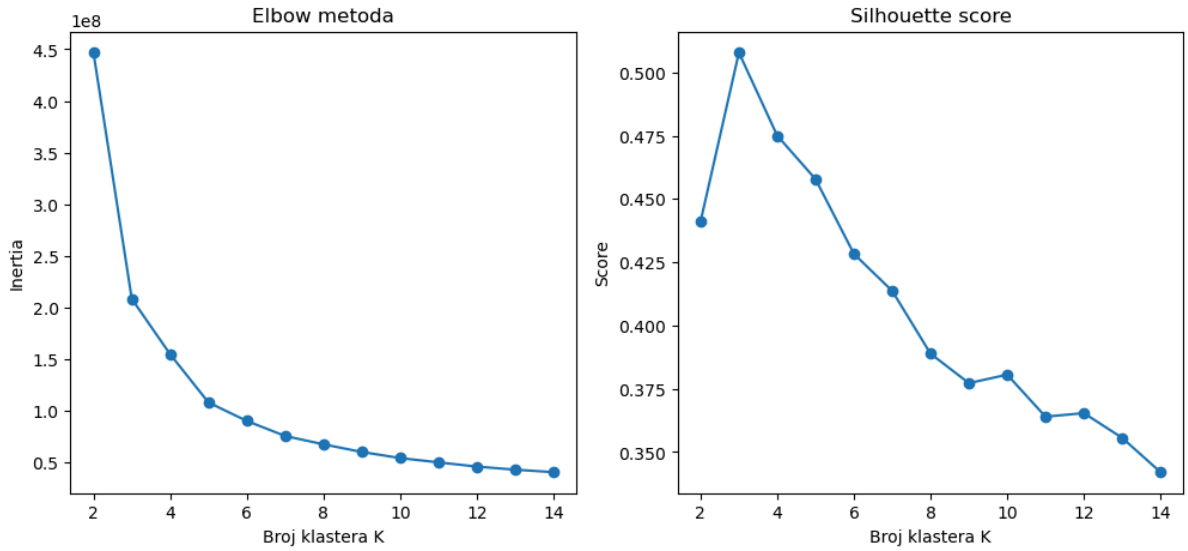
pixels_rgb = np.float32(rakun.reshape(-1, 3))
pixels_cielab = np.float32(rakun_cielab.reshape(-1, 3))

K_range=np.arange(2,15)
inertias, sil_scores, labels_dict = execute_Kmeans(pixels_rgb, K_range)
draw_elbow_and_silhouette(K_range, inertias, sil_scores)
inertias, sil_scores, labels_dict = execute_Kmeans(pixels_cielab, K_rang
draw_elbow_and_silhouette(K_range, inertias, sil_scores)
```

```
2
3
4
5
6
7
8
9
10
11
12
13
14
```



```
2
3
4
5
6
7
8
9
10
11
12
13
14
```

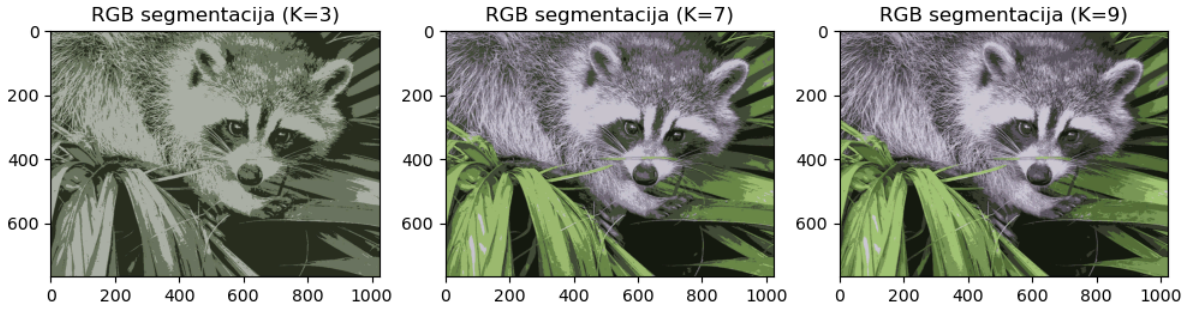


U slučaju slike predstavljene u RGB prostoru boja, sa prikazanih grafika vidimo da su optimalni brojevi klastera 3, 7 ili 9.

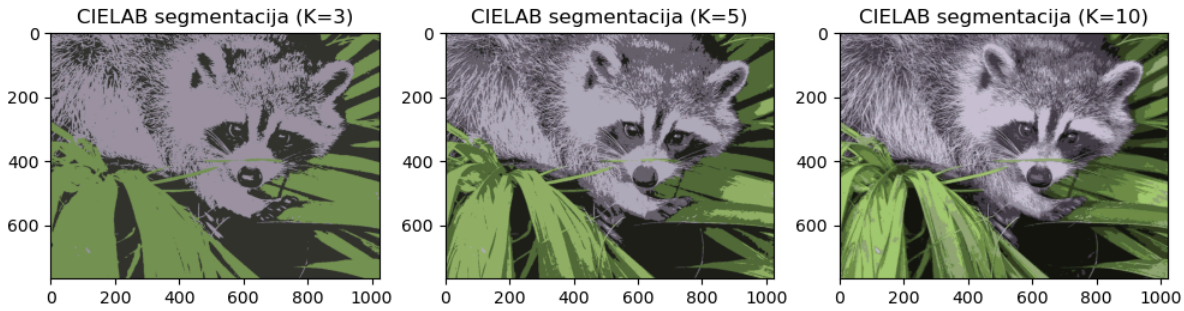
Sa druge strane, ako je slika predstavljena u CIELAB prostoru boja, onda je optimalan broj klastera 3, 5 ili 10.

In [23]:
```
plt.figure(figsize=(12, 6))

# rgb
for i, k in enumerate((3, 7, 9)):
    kmeans_seg = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans_seg.fit(pixels_rgb)
    segmented_img = kmeans_seg.cluster_centers_[kmeans_seg.labels_].asty
    segmented_img = segmented_img.reshape(rakun.shape)
    plt.subplot(1, 3, i+1)
    plt.imshow(segmented_img)
    plt.title(f"RGB segmentacija (K={k})")
```



In [24]:
```
# cielab
plt.figure(figsize=(12, 6))
for i, k in enumerate((3, 5, 10)):
    kmeans_seg = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans_seg.fit(pixels_cielab)
    segmented_lab = kmeans_seg.cluster_centers_[kmeans_seg.labels_].resh
    segmented_lab_rgb = color.lab2rgb(segmented_lab)
    plt.subplot(1, 3, i+1)
    plt.imshow(segmented_lab_rgb)
    plt.title(f"CIELAB segmentacija (K={k})")
```



Za slučaj kada je broj klastera $k = 3$, koji se pokazao kao optimalan u oba prostora boja, segmentacija u CIELAB prostoru dala je prirodnije i lepše boje, ali uz gubitak detalja u strukturi slike (pogotovo na lišću). Nasuprot tome, segmentacija u RGB prostoru za isti broj klastera zadržala je više detalja, ali boje nisu dovoljno izražene. To se dešava zato što sliku sa puno boja pokušavamo da predstavimo pomoću samo 3 boje tako da rezultati ne mogu da budu toliko dobri. U RGB prostoru boja dobili smo otprilike crnu, svetlo i tamno sivu, a u CIELAB crnu, svetlo sivu i zelenu.

Za ostale vrednosti k (RGB: 7 i 9; CIELAB: 5 i 10), rezultati su kvalitetniji tj. više liče na originalnu sliku. Što je veći broj klastera, to je i segmentirana slika bliža originalnoj.

5/5поена

б) У наредном кораку сваком од пиксел-вектора (RGB и CIELAB) додати просторне координате, тј. координате положаја пиксела на слици. То је број врсте и колоне у којој се пиксел налази. На тај начин сваки пиксел постаје елемент петодимензионалног векторског простора. Спровести нову сегментацију слике употребом K-means алгоритма.

(5 поена)

Dodajemo pikselima prostorne koordinate i isprobavamo klasterizaciju samo sa narednim vrednostima $k: 5, 10, 15, 20, 25, 30, 40, 50, 60$ jer bi bilo previše zahtevno sprovoditi klasterizaciju ako bi k uzimalo sve vrednosti od 2 do 60. Zbog toga ne analiziramo optimalan broj klastera pomoću Elbow metode i koeficijenta siluete već ćemo vizuelno analizirati dobijene rezultate.

In [25]:
```
rakun_5d_rgb=np.empty((rakun.shape[0], rakun.shape[1], 5))
rakun_5d_rgb[:,:,:3]=rakun
rakun_5d_rgb[:,:,3:]=np.indices(rakun.shape[:-1]).transpose((1,2,0))
rakun_5d_pixels_rgb = np.float32(rakun_5d_rgb.reshape(-1, 5))

rakun_5d_cielab=np.empty((rakun_cielab.shape[0], rakun_cielab.shape[1],
rakun_5d_cielab[:,:,:3]=rakun_cielab
rakun_5d_cielab[:,:,3:]=np.indices(rakun_cielab.shape[:-1]).transpose((1
rakun_5d_pixels_cielab = np.float32(rakun_5d_cielab.reshape(-1, 5))
```

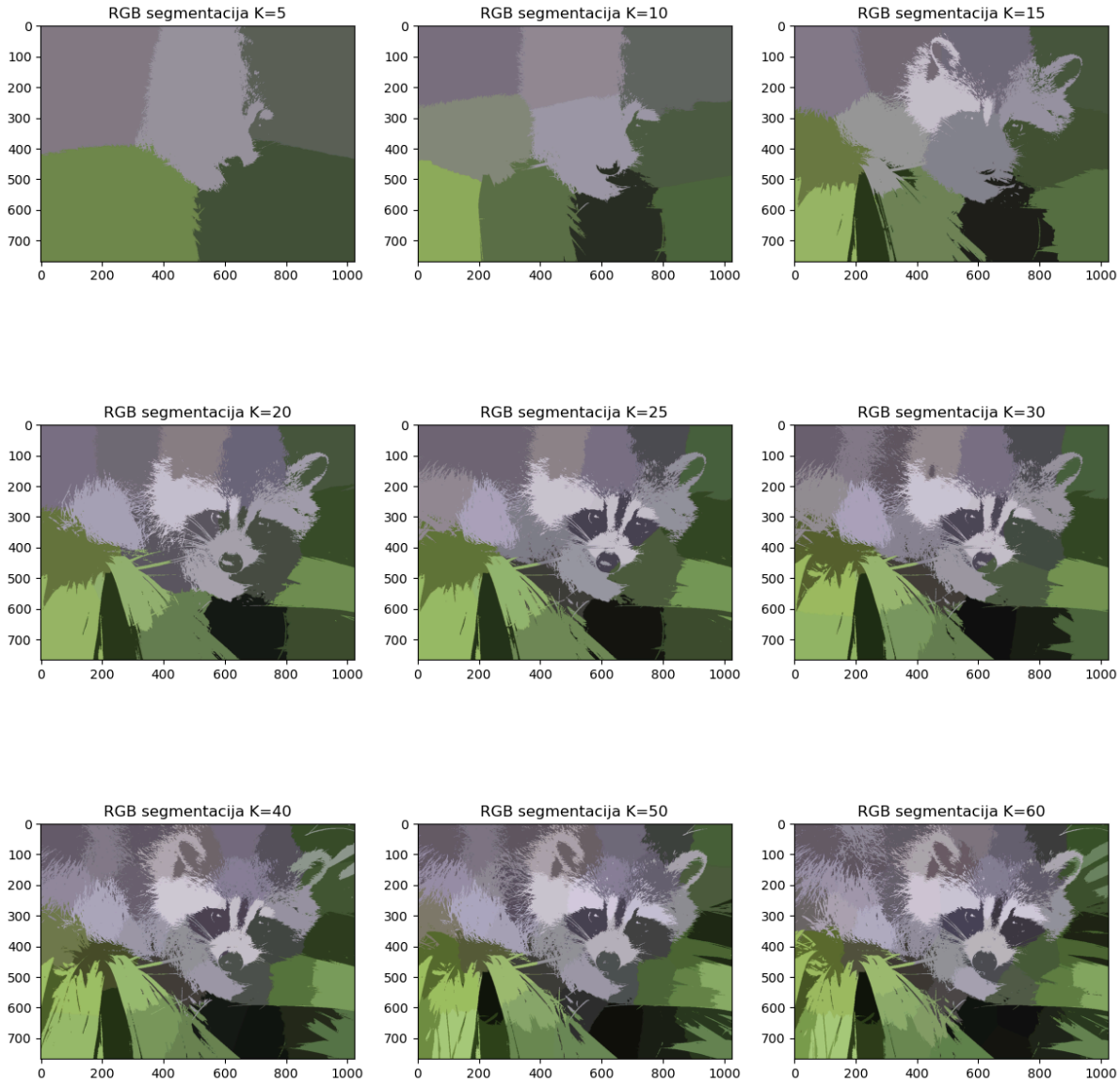
In [26]:

```
K_range=[5, 10, 15, 20, 25, 30, 40, 50, 60]

# RGB
plt.figure(figsize=(15, 16))
for i, k in enumerate(K_range):
    kmeans_seg = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans_seg.fit(rakun_5d_pixels_rgb)
    segmented_img = kmeans_seg.cluster_centers_[kmeans_seg.labels_].asty
    segmented_img = segmented_img[:, :3].reshape(rakun.shape)
    plt.subplot(3, 3, i+1)
    plt.imshow(segmented_img)
    plt.title(f"RGB segmentacija K={k}")
    print(k)

plt.show()
```

```
5
10
15
20
25
30
40
50
60
```


RGB segmentacija K=5
RGB segmentacija K=10
RGB segmentacija K=15
RGB segmentacija K=20
RGB segmentacija K=25
RGB segmentacija K=30
RGB segmentacija K=40
RGB segmentacija K=50
RGB segmentacija K=60

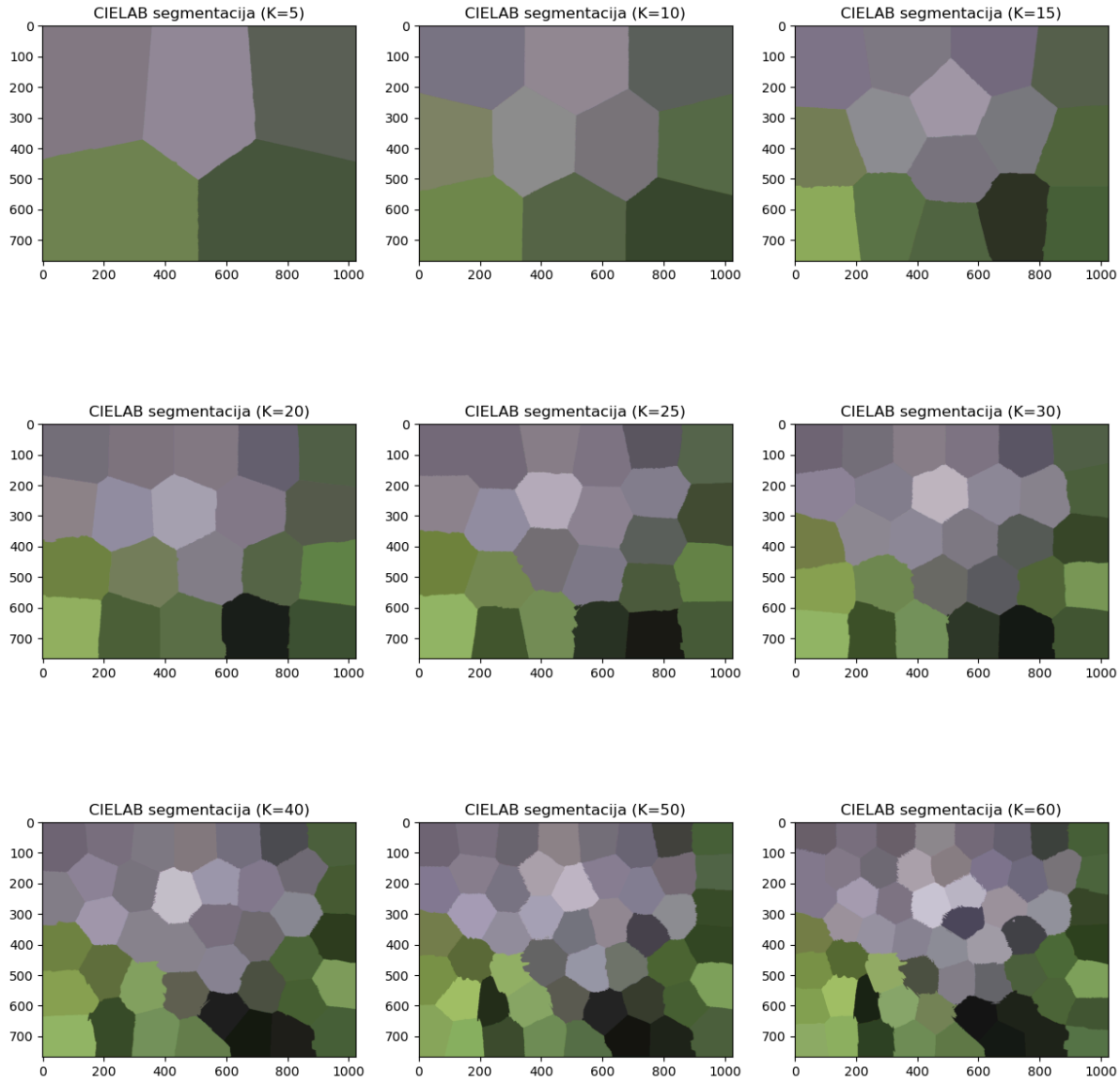
In [27]:

```
plt.figure(figsize=(15, 16))

# CIELAB
for i, k in enumerate(K_range):
    kmeans_seg = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans_seg.fit(rakun_5d_pixels_cielab)
    segmented_lab = kmeans_seg.cluster_centers_[kmeans_seg.labels_]
    segmented_lab = segmented_lab[:, :3].reshape(rakun.shape)
    segmented_lab_rgb = color.lab2rgb(segmented_lab)
    plt.subplot(3, 3, i+1)
    plt.imshow(segmented_lab_rgb)
    plt.title(f"CIELAB segmentacija (K={k})")
    print(k)

plt.show()
```

```
5
10
15
20
25
30
40
50
60
```



Ako posmatramo rezultate RGB segmentacije sa dodatim informacijama o poziciji piksela vidimo da su rezultati dosta bolji u smislu da su grupe piksela koje su međusobno bliske i sličnih boja jasno odvojene, tj. segmentacija je boljeg kvaliteta. Sa povećanjem broja klastera, dobijamo sve više detalja na slici - počinju da se segmentiraju svetliji i tamniji listovi, njuška i oči rakuna itd.

Sa druge strane, kada koristimo CIELAB prostor boja dobijamo mnogo manje detalja pri segmentaciji. Za mali broj klastera (npr. $k = 5$) segmentacija je slična kao kod RGB s tim što u CIELAB prostoru boja granice između klastera su pravolinijske. Za veliki broj klastera ne dobijamo jasne detalje kao kod RGB segmentacije, već skup homogenih površina sa jasnijim ivicama nego kod RGB.

5/5поена

в) У наредном кораку скалирати све компоненте на интервал $[0, 1]$ користећи информације о минимуму и максимуму дуж одговарајуће координатне осе.

(5 pasus)

In [28]:
```
plt.imshow(rakun);
```



In [29]:
```
from sklearn.preprocessing import MinMaxScaler
#dva odvojena scaler objekta da bismo kasnije mogli da uradimo inverse_t
scaler_rgb = MinMaxScaler()
scaler_cielab = MinMaxScaler()
rakun_5d_rgb_scaled = scaler_rgb.fit_transform(rakun_5d_pixels_rgb)
rakun_5d_cielab_scaled = scaler_cielab.fit_transform(rakun_5d_pixels_cie
```

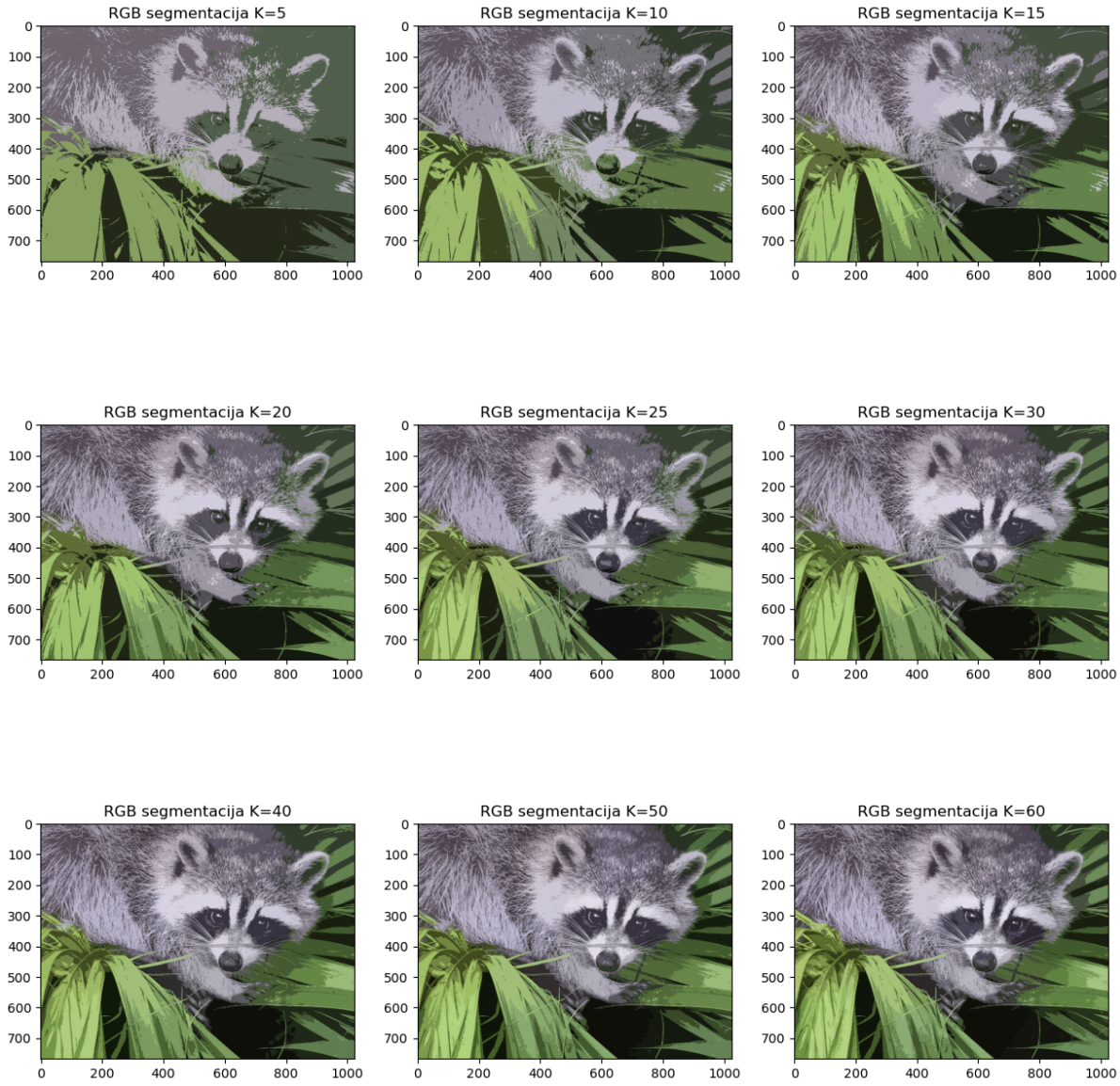
In [30]:

```
K_range=[5, 10, 15, 20, 25, 30, 40, 50, 60]

# RGB
plt.figure(figsize=(15, 16))
for i, k in enumerate(K_range):
    kmeans_seg = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans_seg.fit(rakun_5d_rgb_scaled)
    segmented_img = kmeans_seg.cluster_centers_[kmeans_seg.labels_]
    segmented_img = np.uint8(scaler_rgb.inverse_transform(segmented_img))
    segmented_img = segmented_img[:, :3].reshape(rakun.shape)
    plt.subplot(3, 3, i+1)
    plt.imshow(segmented_img)
    plt.title(f"RGB segmentacija K={k}")
    print(k)

plt.show()
```

```
5
10
15
20
25
30
40
50
60
```



Vidimo da su sada rezultati još bolji u smislu da slika dosta više liči na original. Kad podaci nisu bili skalirani, komponente vektora koje su određivale boju su se kretale u opsegu od 0 do 255, a komponente koje su određivale poziciju piksela u opsegu od 0 do 767 i od 0 do 1023. Praktično, informacije o poziciji piksela su bile favorizovane od strane algoritma jer su imale veći opseg u odnosu na komponente boje. Zbog toga su i dobijeni klasteri bili mnogo više zavisni od pozicije piksela, a sada kada su vektori normalizovani, svaka komponenta (i boja i pozicija piksela) se gledaju ravnomerno, pa dobijamo rezultate koji više podsećaju na originalnu sliku i bliži su rezultatima kada je rađena RGB segmentacija bez informacije o poziciji piksela.

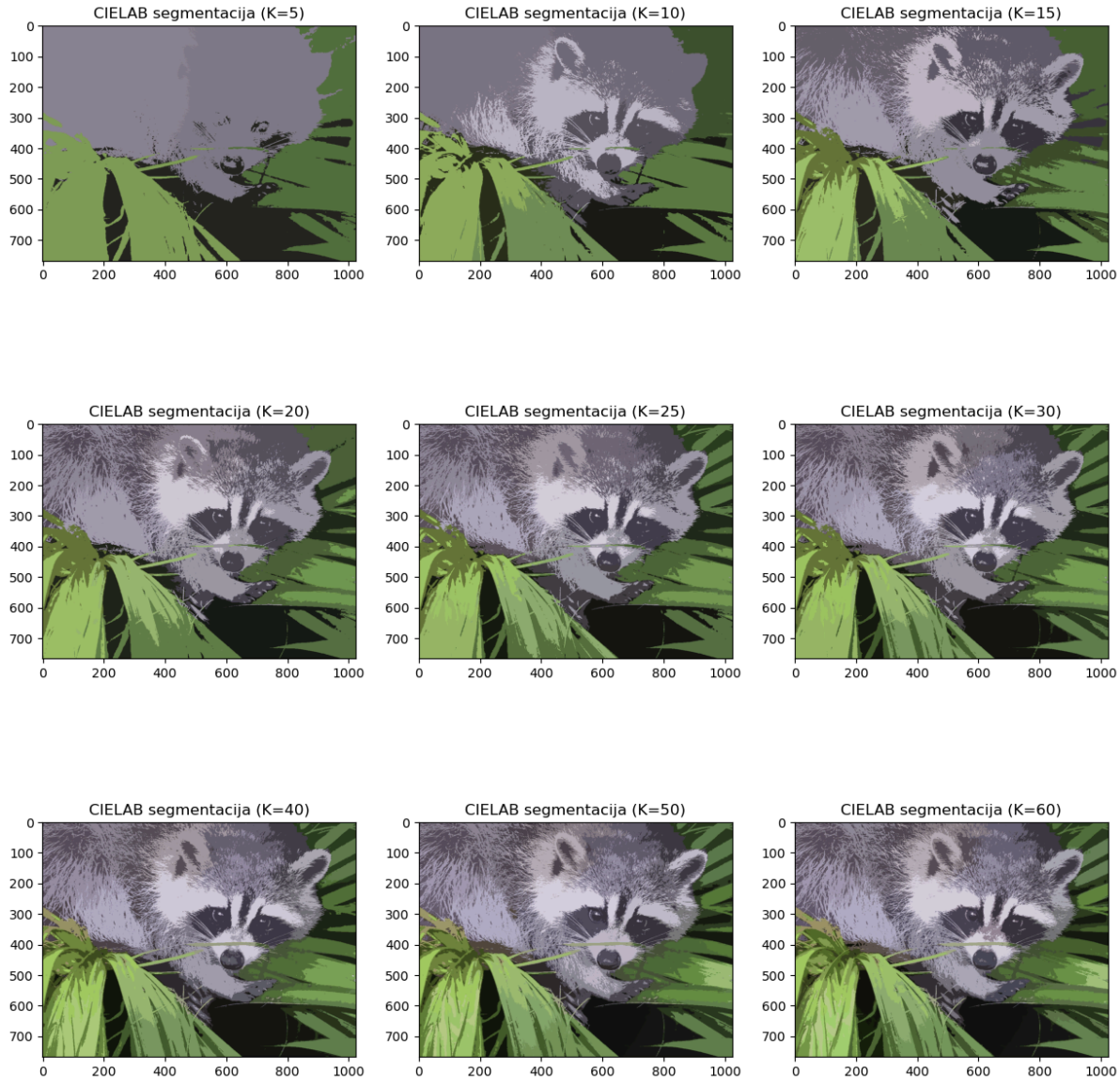
In [31]:

```
plt.figure(figsize=(15, 16))

# CIELAB
for i, k in enumerate(K_range):
    kmeans_seg = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans_seg.fit(rakun_5d_cielab_scaled)
    segmented_lab = kmeans_seg.cluster_centers_[kmeans_seg.labels_]
    segmented_lab = scaler_cielab.inverse_transform(segmented_lab)
    segmented_lab = segmented_lab[:, :3].reshape(rakun.shape)
    segmented_lab_rgb = color.lab2rgb(segmented_lab)
    plt.subplot(3, 3, i+1)
    plt.imshow(segmented_lab_rgb)
    plt.title(f"CIELAB segmentacija (K={k})")
    print(k)

plt.show()
```

```
5
10
15
20
25
30
40
50
60
```



Ono što se i ovde može videti je da klasterizacija sa CIELAB prostorom boja daje manje detalja i u slučaju kad su podaci skalirani. Na primer, za $K = 5$ kod RGB segmentacije se vide detalji na krznu rakuna, dok kod CIELAB nema toga. Čak i kad je broj klastera veliki (npr. $K = 60$) kod CIELAB segmentacije možemo da vidimo neke delove na slici (npr. deo između njuške i očiju rakuna) koji su prikazani kao homogena površina, a isti taj deo slike kod RGB klasterizacije ima više detalja i "mekši" prikaz.

5/5поена

г) Искористити претходни поступак сегментације за компресију учитане слике. Сваки тродимензионални пиксел-вектор заменити центром кластера коме тај пиксел припада.

(5 поена)

Заменом сваког пиксела центром његовог кластера вршимо компресију.
Проблем се може посматрати као складиштење матрица.

Оригинални формат

$$N_{\text{pix}} \times 3 \quad \text{(float/int)}$$

Компресовани формат

- Матрица индекса:

$$N_{\text{pix}} \times 1 \quad \text{(int)}$$

- Матрица центара (Codebook):

$$K \times 3 \quad \text{(float)}$$

Segmentiraćemo opet originalnu sliku rakuna za $k = 9$ klastera jer se to pokazalo kao najbolji rezultat u delu $4a$ na osnovu analize inercije i koeficijenta siluete. Naravno, ako bismo još više povećali broj klastera (npr. na $k = 14$) dobili bismo još bolje rezultate, tj. slika bi bila bliža originalnoj jer bismo imali 14, a ne 9 boja, dok bi se potreban prostor neznatno povećao (konkretno matrica centara bi umesto 9×3 bila dimenzija 14×3).

In [32]:

```
pixels_rgb = rakun.reshape((-1, 3))
K_values = [9, 14]
labels={}
centers={}

plt.figure(figsize=(12, 6))

for i, k in enumerate(K_values):
    kmeans_seg = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans_seg.fit(pixels_rgb)

    # matrica indeksa (Npix x 1)
    labels_matrix = kmeans_seg.labels_.reshape(-1, 1)
    labels[i]=labels_matrix

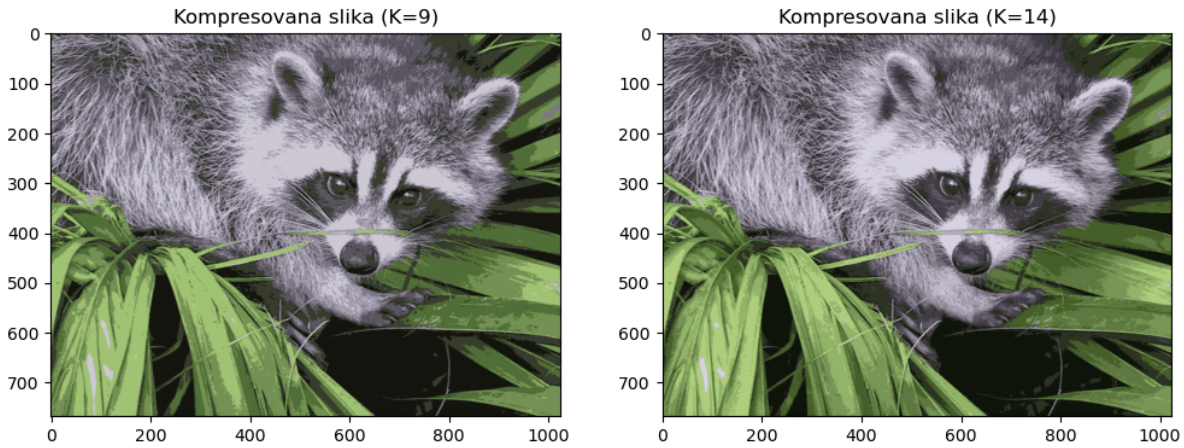
    # matrica centara (K x 3)
    centers_matrix = kmeans_seg.cluster_centers_
    centers[i]=centers_matrix

    print(f"K={k}")
    print("Matrica indeksa:", labels_matrix.shape)
    print("Matrica centara:", centers_matrix.shape)

    # rekonstrukcija slike
    compressed_img = centers_matrix[kmeans_seg.labels_]
    compressed_img = compressed_img.reshape(rakun.shape)
    plt.subplot(1, 2, i+1)
    plt.imshow(compressed_img.astype(np.uint8))
    plt.title(f"Kompresovana slika (K={k})")

plt.show()
```

```
K=9
Matrica indeksa: (786432, 1)
Matrica centara: (9, 3)
K=14
Matrica indeksa: (786432, 1)
Matrica centara: (14, 3)
```



5/5поена

д) Израчунати уштеду меморије и приказати однос компресије за:

$$K \in \{2, 5, 10, 20\}.$$

(5 поена)

In [33]:
```
print(f"Originalna slika: {rakun.size * rakun.itemsize / 1024 / 1024} MB
```

```
Originalna slika: 2.25 MB
```

Za matricu indeksa tip podataka koji je dovoljan je `uint8` jer sa njim možemo da imamo 256 različitih vrednosti, a nama konkretno ovde treba 9 ili 14. Za matricu centara takođe možemo koristiti ovaj tip podataka jer su nam centri zapravo RGB vektori čije komponente možemo pretvoriti u `uint8` tip jer se kreću u opsegu od 0 do 255.

In [34]:
```
Npix = pixels_rgb.shape[0]
original_size_bytes = rakun.size * rakun.itemsize

for K in [2, 5, 10, 20]:
    compressed_size_bytes = (Npix * np.dtype(np.uint8).itemsize) + (K *
    ratio = original_size_bytes / compressed_size_bytes
    save = 1 - compressed_size_bytes / original_size_bytes
    print(f"K={K}, Odnos kompresije: {ratio:.2f}, Ušteda memorije: {save
```

```
K=2, Odnos kompresije: 3.00, Usteda memorije: 66.67% (1.50 MB), Nova ve
licina: 0.75 MB
K=5, Odnos kompresije: 3.00, Usteda memorije: 66.67% (1.50 MB), Nova ve
licina: 0.75 MB
K=10, Odnos kompresije: 3.00, Usteda memorije: 66.67% (1.50 MB), Nova v
elicina: 0.75 MB
K=20, Odnos kompresije: 3.00, Usteda memorije: 66.66% (1.50 MB), Nova v
elicina: 0.75 MB
```

Odnos kompresije je uvek približno jednak 3 jer je matrica indeksa dominantna iako zauzima 3 puta manje prostora od originalne slike (u originalnoj slici jedan piksel je zauzimao 3 bajta, a sada zauzima po 1 bajt). Matrica centara je u odnosu na matricu indeksa mnogo manja, pa skoro i da ne utiče na odnos kompresije.

5/5поена

Укупно 100/100поена